

PORTADA

Técnicas de Programación

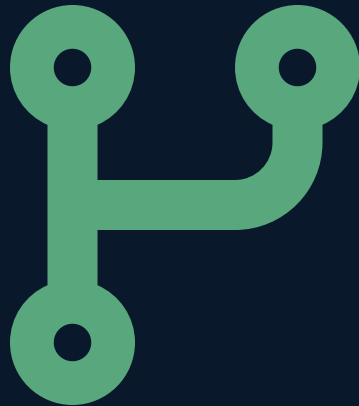
Unidad 2 — Paradigma Objetual: Sobrecarga de Métodos

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Mismo nombre. Distinta lista de parámetros.
Java decide cuál ejecutar antes de correr una sola línea.**



TRANSICIÓN

Recordando la sesión anterior

Ya viste que una clase puede tener **varios constructores implícitos según lo que necesites** — pero solo mencionamos uno por clase. ¿Qué pasa si quieres construir un `Estudiante` a veces con código, y a veces sin él todavía asignado? ¿O un método `sumar` que sirva tanto para enteros como para decimales, sin escribir dos nombres distintos?

CONCEPTO

Sobrecarga de métodos (*overloading*)

La **sobrecarga de métodos** permite que una clase tenga **varios métodos con el mismo nombre**, siempre que se distingan por su **firma**: la cantidad, el tipo o el orden de sus parámetros.

```
public int sumar(int a, int b) {  
    return a + b;  
}  
  
public double sumar(double a, double b) {  
    return a + b;  
}
```

Java decide automáticamente cuál de las dos versiones ejecutar **según los argumentos con los que se llame al método**:
sumar(2, 3) usa la primera; sumar(2.5, 3.1) usa la segunda.

CONCEPTO

¿Qué cuenta como firma distinta?

La **firma** de un método es su nombre más la lista de tipos de sus parámetros (no incluye el tipo de retorno ni los nombres de los parámetros). Dos métodos están sobrecargados si tienen el mismo nombre pero firmas distintas.

Cambio	¿Es sobrecarga válida?
Distinta cantidad de parámetros	Sí
Mismo número, distinto tipo de parámetros	Sí
Mismos parámetros, distinto tipo de retorno únicamente	No — error de compilación

CONTRAEJEMPLO

El error más común: cambiar solo el retorno

```
public int sumar(int a, int b) { return a + b; }  
  
// Error de compilación: misma firma que la de arriba,  
// el tipo de retorno no cuenta para distinguir la sobrecarga  
public double sumar(int a, int b) { return a + b; }
```

El **compilador rechaza esto** porque, al llamar `sumar(2, 3)`, no hay forma de saber cuál de los dos métodos se quiso invocar — el tipo de retorno no participa en la decisión.

CONCEPTO

Constructores sobrecargados

Los constructores también se sobrecargan: una clase puede ofrecer varias formas de construir un objeto, según qué datos estén disponibles al momento de crearlo.

```
public class Estudiante {  
    private String nombre;  
    private String codigo;  
  
    public Estudiante(String nombre) {  
        this(nombre, "SIN-ASIGNAR");    // delega en el otro constructor  
    }  
  
    public Estudiante(String nombre, String codigo) {  
        this.nombre = nombre;  
        this.codigo = codigo;  
    }  
}
```

`this(...)` (distinto de `this.`) llama a **otro constructor de la misma clase** — evita repetir la misma lógica de inicialización dos veces.

CONCEPTO

Un caso real: `System.out.println`

La sobrecarga no es un tecnicismo aislado: la usas desde la primera semana del curso sin saberlo. La clase `PrintStream` de Java define **más de diez versiones sobrecargadas** de `println`, una por cada tipo de dato.

```
System.out.println(42);           // println(int)
System.out.println(3.14);         // println(double)
System.out.println("Hola");       // println(String)
System.out.println(true);         // println(boolean)
```

Cada llamada resuelve a un método distinto, elegido por el tipo del argumento que le pasaste.

TENSIÓN CONCEPTUAL

Sobrecarga (*overloading*) vs. sobrescritura (*overriding*)

Son dos mecanismos distintos que suenan parecido en español y se confunden con frecuencia — no son intercambiables.

	Sobrecarga (<i>overloading</i>)	Sobrescritura (<i>overriding</i>)
¿Dónde ocurre?	Varios métodos en la misma clase	Un método en una subclase que redefine el de su superclase
¿Qué cambia?	La firma (parámetros)	La implementación, misma firma
¿Cuándo se decide?	En tiempo de compilación	En tiempo de ejecución

La **sobrescritura** (`@Override`) ya la viste en la sesión pasada con `Persona.saludar()` ; la sobrecarga de hoy es un mecanismo distinto, aunque comparta la idea de "un mismo nombre, más de un comportamiento".

CONCEPTO

Cuándo Java no puede decidir: ambigüedad

Si dos versiones sobrecargadas son igual de "cercanas" a los argumentos de la llamada (por ejemplo, mezclando tipos que Java puede convertir automáticamente entre sí), el compilador no logra elegir una y marca un **error de ambigüedad**, no un error de ejecución.

```
public void mostrar(int a, double b) { }  
public void mostrar(double a, int b) { }  
  
mostrar(1, 2); // Error: ambiguo – ¿1 se convierte a double,  
               // o 2 se convierte a double? Java no elige por ti.
```

Diseñar sobrecargas con firmas claramente distintas evita este problema desde el inicio.

INTERACCIÓN

¿Cuál versión se ejecuta?

Dada esta clase, indiquen qué método se ejecuta en cada llamada (o si hay error):

```
void mostrar(int a) { }  
void mostrar(int a, int b) { }  
void mostrar(String a) { }
```

1. mostrar(5)
2. mostrar(5, 10)
3. mostrar("hola")
4. mostrar(5.0)

5 minutos · discutan en parejas el caso 4 con cuidado

SÍNTESIS

Lo que hay que llevarse de hoy

1. La sobrecarga permite varios métodos (o constructores) con el mismo nombre, distinguidos por su firma: cantidad, tipo u orden de parámetros.
2. El tipo de retorno **no** distingue una sobrecarga — usarlo como único diferenciador es error de compilación.
3. `this(...)` delega en otro constructor de la misma clase para no repetir lógica de inicialización.
4. Sobrecarga (overloading) y sobrescritura (overriding) son mecanismos distintos: la primera se resuelve en compilación según la firma; la segunda, en ejecución según la subclase del objeto.



CIERRE

Antes de la próxima sesión

- Escribir en el IDE al menos tres versiones sobrecargadas de un mismo método propio y comprobar en cada llamada cuál se ejecuta.
- Repasar la diferencia entre `private` , `protected` y `public` vista la sesión pasada — se retoma al hablar de clases abstractas.

La próxima sesión sigue dentro de la Unidad 2 con clases abstractas y el modificador `static` .

